

Internship Progress

aurelien astesiano

April 2026

1 Code

1.1 Parameters

- u_0 (1.0) : Self-propulsion speed
- r_R (0.075) : Agent-agent repulsion radius, below which agents repel each other.
- r_A (0.1) : Maximum distance at which an agent aligns with other agents.
- ν (2.0) : Alignment intensity (in front of the target vector).
- τ (0.15) : Distance below which an agent starts to feel the presence of an obstacle and repel it.
- C_ϕ (5.0) : Agent-obstacle repulsion intensity
- d_s (0.02) : Noise on orientation
- η (1.0) : Viscosity felt by the obstacles
- d_o (0.0) : Noise on obstacles set to 0 by default
- κ **variable** (10, 100 or 1000) : Obstacle spring stiffness.
- μ **variable** Agent-agent repulsion intensity.
- ζ **variable** Viscosity on agents.
- M Number of agents set to 3000 by default
- N Number of obstacles set to 3000 by default
- L Length of an edge set to 1 by default
- dt Discretization in the Euler scheme, set to 0.005 by default but we will implement an adaptive time step (see below)

1.2 Grid

We consider the maximum interaction radius of an agent with its surroundings, which is the maximum between r_a , r_r , and τ . We divide the total size by this parameter to get the number of cells.

We create an array that counts the number of elements per grid cell and an array that gives the indices of the elements in each cell of the grid.

We fill it with a loop over the agents; for each, we take the integer part of their position divided by the cell size. We then get the cell in which the agent is located, and we add 1 to the corresponding counter for that cell. We do the same for the obstacles.

1.3 Forces

We start with the agent-obstacle potential:

$$\phi(x) = \frac{3C_\phi}{2\pi\tau} \left(1 - \frac{|x|}{\tau}\right)_+^2$$

which gives the force gradient applied each time:

$$F_\phi(x) = -\nabla\phi(x) = \frac{3C_\phi}{\pi\tau^2} \left(1 - \frac{|x|}{\tau}\right) \frac{\vec{x}}{|x|}$$

Thus, in the code, we keep the prefactor constant in memory. Same thing for the agent-agent repulsion:

$$\psi(x) = \frac{6\mu}{\pi r_R^2} \left(1 - \frac{|x|}{r_R}\right)_+^2$$

Which gives the inter-agent force:

$$F_\psi(x) = -\nabla\psi(x) = \frac{12\mu}{\pi r_R^3} \left(1 - \frac{|x|}{r_R}\right) \frac{\vec{x}}{|x|}$$

and we keep the prefactor in memory.

1.4 Hooke's Spring Stability

Since the force can be very strong (stiff spring), we use an exponential Euler scheme, for which the proof of the formula used in the code is given here:

Let $U(t) = X_i(t) - Y_i$ be the distance from the obstacle to its anchor point and $a = \frac{\kappa}{\eta}$. Assuming the external force F_{ext} is constant over the interval $[t, t + \Delta t]$ (which is indeed the case in our discretization), the differential equation for the obstacles is written as:

$$\frac{dU}{dt} = -aU(t) + F_{ext}$$

Multiplication by the integrating factor e^{at} :

$$e^{at} \frac{dU}{dt} + ae^{at}U(t) = e^{at}F_{ext}$$

$$\frac{d}{dt}(U(t)e^{at}) = e^{at}F_{ext}$$

Integration over the interval $[t, t + \Delta t]$:

$$U(t + \Delta t)e^{a(t+\Delta t)} - U(t)e^{at} = F_{ext} \int_t^{t+\Delta t} e^{as} ds$$

$$U(t + \Delta t)e^{a(t+\Delta t)} - U(t)e^{at} = F_{ext} \frac{e^{a(t+\Delta t)} - e^{at}}{a}$$

Multiplication by $e^{-a(t+\Delta t)}$ to isolate $U(t + \Delta t)$:

$$U(t + \Delta t) = U(t)e^{-a\Delta t} + F_{ext} \frac{1 - e^{-a\Delta t}}{a}$$

Deduction of the exact displacement of the obstacle over the time step, $\Delta X_i = U(t + \Delta t) - U(t)$:

$$\Delta X_i = U(t)(e^{-a\Delta t} - 1) + F_{ext} \frac{1 - e^{-a\Delta t}}{a}$$

The factors in front of $U(t)$ and F_{ext} are calculated at initialization in the code; then, to obtain the displacement of an obstacle during Δt , we apply the formula above. The external forces being the obstacle-agent interactions.

1.5 Angle Variation

In the code, we use the angle value on the trigonometric circle rather than the vector. Using the fact that:

$$J_k = \sum_{|Z_k - Z_j| \leq r_A} \alpha_j = \left(\frac{\sum \cos \theta_j}{\sum \sin \theta_j} \right)$$

and

$$\tan(\bar{\theta}_k) = \frac{Y}{X} = \frac{\sum \sin \theta_j}{\sum \cos \theta_j}$$

To find the mean theta, we use the appropriate Python function `arctan2`, which avoids non-bijection and division by 0 issues; it is defined as follows:

$$\arctan2(Y, X) = \begin{cases} \arctan(\frac{Y}{X}) & \text{if } X > 0 \\ \arctan(\frac{Y}{X}) + \pi & \text{if } X < 0 \text{ and } Y \geq 0 \\ \arctan(\frac{Y}{X}) - \pi & \text{if } X < 0 \text{ and } Y < 0 \\ \frac{\pi}{2} & \text{if } X = 0 \text{ and } Y > 0 \\ -\frac{\pi}{2} & \text{if } X = 0 \text{ and } Y < 0 \end{cases}$$

Once we have this angle, we apply the orthogonal projection in the following way:

$$\nu \bar{\alpha}_k \cdot \alpha_k^\perp = \nu \begin{pmatrix} \cos \bar{\theta}_k \\ \sin \bar{\theta}_k \end{pmatrix} \cdot \begin{pmatrix} -\sin \theta_k \\ \cos \theta_k \end{pmatrix}$$

$$\nu \bar{\alpha}_k \cdot \alpha_k^\perp = \nu (\sin \bar{\theta}_k \cos \theta_k - \cos \bar{\theta}_k \sin \theta_k) = \nu \sin(\bar{\theta}_k - \theta_k)$$

The calculation above is literally translated into the code.

1.6 Adaptive dt

To be able to support an adhesive force that will create high concentrations, it is necessary to add an adaptive time step. For this, we calculate the maximum speed of all agents during this iteration (this is the last step of the loop, all forces having been calculated beforehand) and we arbitrarily set:

$$dt_{ideal} = \frac{0.3 * r_{min}}{v_{max}}$$

Where r_{min} is the minimum interaction radius so that with this dt we cannot "jump through" an interaction.

In this case, we choose a fraction of time to advance that is lower than dt_{ideal} . Since the simulation sometimes explodes in complexity, we add dividing the time step by 10 as an absolute minimum limit.

With this compromise, the simulations do not take too much time and have a coherent output.

1.7 Tip

For agent-agent interactions (repulsion and alignment), since they are symmetric, we only look at the top and right cells and process the double interaction directly, which halves the computation time for these forces.

The while loop and the time as a parameter of the function allow, during display, to simply advance by the precisely desired time, and the function will reach it, making it possible to record at regular intervals.

2 Adhesive Case

To switch to the adhesive case, we simply add a minus in front of the agent-obstacle repulsion force. We can later discuss the relevance of this new force, as it stability issues when the distance between an agent and an obstacle approaches 0.

2.1 Micro Simulation Results

We will try to reproduce the results from the article but in the adhesive case with the same parameters. In the phase diagrams, we vary the agent-agent

repulsion (μ), which increases from left to right, and the viscosity on the agents (ζ), which increases from bottom to top.

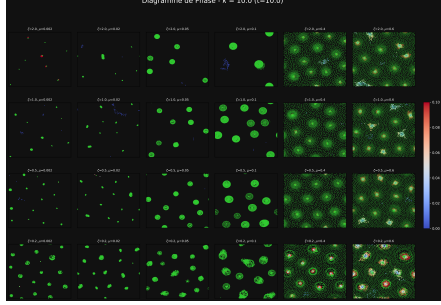


Figure 1: Simulation (Adhesive model) - $\kappa = 10$

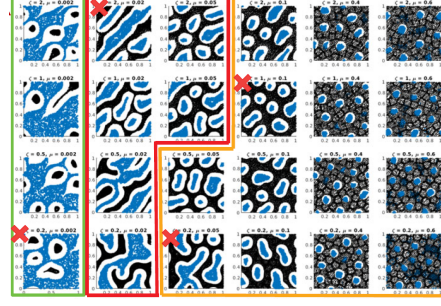


Figure 2: Article - $\kappa = 10$, $\zeta = [2.0, 1.0, 0.5, 0.2]$, $\mu = [0.002, 0.02, 0.05, 0.1, 0.4, 0.6]$

Analysis ($\kappa = 10$): Here we can clearly see the split between the first 4 columns, from which clusters can emerge, and the next two, where this is no longer possible. Only the cluster size varies, and only the case zeta = 2 and mu = 0.05 shows 2 moving clusters. A parallel with honeycombs can be seen in the last two columns, even though the obstacles are already concentrated in the previous ones.

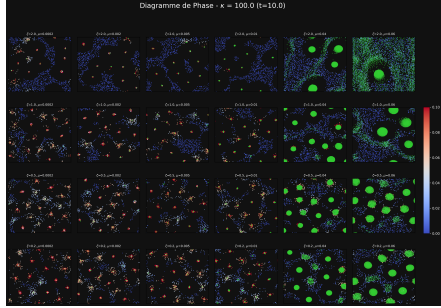


Figure 3: $\kappa = 100$, cells are green and obstacles range from blue to red through white depending on the distance to their anchor point

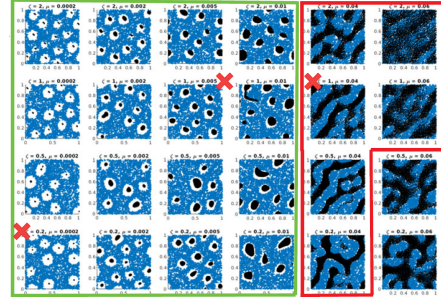


Figure 4: Article - $\kappa = 100$, $\zeta = [2.0, 1.0, 0.5, 0.2]$, $\mu = [0.0002, 0.002, 0.005, 0.01, 0.04, 0.06]$

Analysis ($\kappa = 100$): Here the cells gather tightly around fixed but unstable concentrated points, so some can move until they find an equilibrium point depending on the distribution of surrounding obstacles. In the last two columns, there are still cells that have a track movement following the obstacles; however, clusters still emerge (strong predominance of clusters in the bottom right, coexistence in the top right). In the top right, they are no longer clusters that suck

in particles forever, but rather particles enter and leave them via self-propulsion.

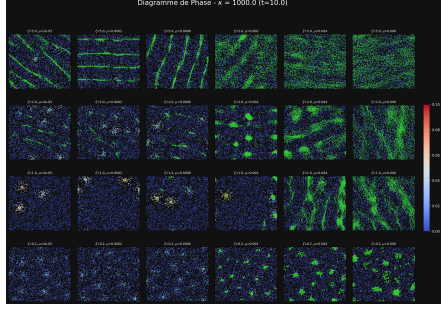


Figure 5: $\kappa = 1000$

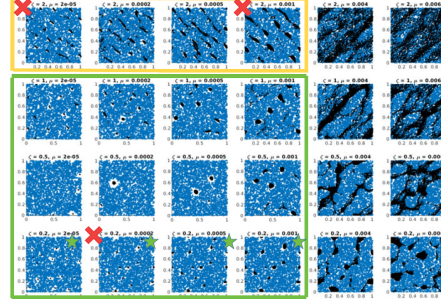


Figure 6: Article - $\kappa = 1000$, $\zeta = [5.0, 2.0, 1.0, 0.2]$, $\mu = [2e-5, 2e-4, 6e-4, 2e-3, 4e-3, 6e-3]$

Analysis ($\kappa = 1000$): In this case, the observed behavior is extremely similar. On the first row, the clusters are fixed as in the article; however, this occurs across the entire row, but in the last two, some cells are fixed and others are moving. Above, the clusters are moving, so the results are really very similar.

2.2 Future Developments

- Numerical instability: For now, with the lower limit on the time step, we maintain a local instability. To resolve it, we will try to remove the lower bound on the time step and add a condition that when the distance between the particle and the obstacle is extremely small (less than $10e^{-12}$), the force is zero. I will run tests again and recreate a phase diagram to see if we obtain differences and if the simulations are coherent.
- Once the instabilities are resolved, we can move towards a more advanced comparison between adhesion and repulsion. The analysis would focus on when differences arise between adhesion and repulsion. To do this, I will first understand and reuse the geometrically invariant quantifiers used in the article to analyze the results. Once this is done, it will be necessary to determine the regions to test and the questions we want to answer.
- This ties into the point above, but there is also an avenue to explore regarding the initial distribution of obstacles; we can imagine distributions in bands, spheres, or other shapes and observe the differences.

3 Comparison adhesive/repulsive

3.1 Choice of the distance

We will focus here on the different type of distances that can be used to observe patterns differences (or not) between to images. Then we will compare them over a data set.

3.1.1 Wasserstein 1D

As done in the article we can take two discrete distributions, project them on a grid with same size (we will talk about optimal grid size later) and compare the wasserstein distance of there signatures. Making it totally space invariante over a data set.

3.1.2 Fourier

to have a distance considering geometric we can think of using fourier transform of our density, the intuition is that it will be very efficient to see difference between bandes and clusters for exemple. We put our two discrete models on a thin grid (also discussion on the choice of the grid) and we take the spectrum of this density. To compare two densities we can take L2 difference of their spectrum.

3.1.3 Topological data analysis

Following method is explained in two pages in the section 3 of [3] The objective is to map a point cloud to a mathematical vector characterizing its global shape. Based on an article that directly applied this framework to active matter, we explain the general principle.

We start with a point cloud consisting of coordinates $X = \{r_1, r_2, \dots, r_K\}$ in a 2D space.

Mathematically, we define building blocks called *simplices*:

- A 0-simplex is a point.
- A 1-simplex is a line segment connecting two points.
- A 2-simplex is a solid triangle (composed of 3 points).

The Connection Rule (Filtration) We imagine a ball of diameter ϵ around each point. We gradually increase this scale parameter ϵ starting from 0:

- If two balls overlap (meaning the distance between the two points is less than the defined threshold), we create an edge (a 1-simplex).
- If three edges form a closed triangle, we fill it to create a face (a 2-simplex).

We then analyze two homology groups:

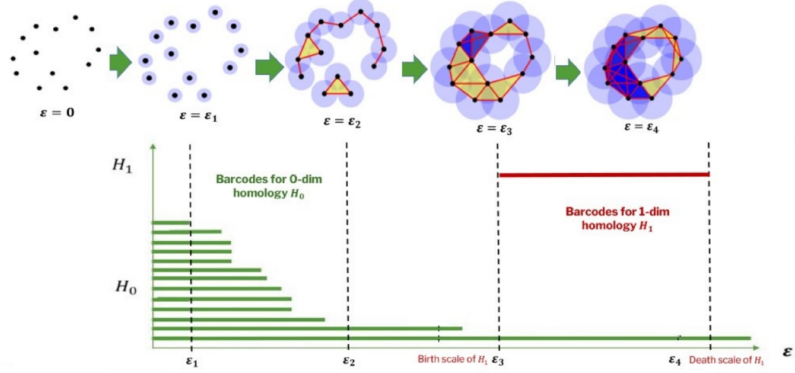


Figure 7: filtration

- **The H_0 group:** It represents the connected components (clusters or islands).
- **The H_1 group:** It represents 1D holes (loops). A hole is "born" when edges form a closed cycle without a solid face inside, and it "dies" when ϵ is large enough for faces (triangles) to completely fill this empty space.

Persistence (The Barcode) For each topological feature (an H_0 island or an H_1 hole), the algorithm records two fundamental values:

- Its **birth** (b) at a given scale ϵ .
- Its **death** (d) when the feature disappears or merges with a larger one.
- Its **persistence** $P = d - b$, which measures the "lifespan" or robustness of this topological feature.

Persistence Landscapes Next, we must transform these "barcodes" into exploitable data. To do so, "tent functions" are used. Considering continuous values of the scale parameter ϵ , these functions are defined for each interval (b, d) as:

$$h_{(b,d)}(\epsilon) = \max \left\{ 0, \frac{P}{2} - \left| \epsilon - \frac{b+d}{2} \right| \right\} \quad (1)$$

From these functions, we define the m -th upper envelope, which corresponds to the m -th largest value at a given ϵ (see figure). This yields a persistence landscape composed of $\{\lambda_1, \lambda_2, \dots, \lambda_M\}$. The integer M is deliberately chosen to be relatively small to filter out topological noise, keeping only the features that persist long enough (i.e., tall tent functions).

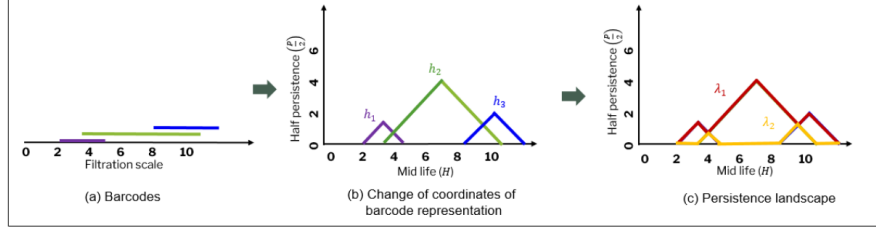


Figure 8: barcode and landscapes

We can then compute the persistence landscape contour (or average landscape):

$$L(\epsilon) := \frac{1}{M'} \sum_{m=1}^{M'} \lambda_m(\epsilon) \quad (2)$$

Using theoretical results remained in [3], $L(\epsilon)$ is proven to be a continuous function on a bounded interval $[0, \epsilon_{max}]$, belonging to a normed vector space (the L^p space). This provides the mathematical justification to treat these landscapes as standard Euclidean vectors.

Vectorization and Comparison In practice, this function is not evaluated continuously. We define a regular grid of resolution K over the studied interval, such that $\epsilon_k = k \cdot \Delta\epsilon$ for $k \in \{1, \dots, K\}$. The continuous function $L(\epsilon)$ is sampled to become a vector in $\mathbb{R}^K$:

$$\vec{V} = [L(\epsilon_1), L(\epsilon_2), \dots, L(\epsilon_K)] \quad (3)$$

At this stage, the reference article performs a series of statistical tests tailored to their specific objectives. However, for our specific goal of comparing two distinct simulations (e.g., simulation A and B), we can simply compute the Euclidean difference:

$$\text{Distance} = \|\vec{V}_A - \vec{V}_B\|_2 \quad (4)$$

This topological distance can then be plotted on a graph as a function of each physical parameter we wish to investigate.

Adaptation : Periodic Cubical Homology on Density Fields We change the filtration because VR seems to not work, this method is too sensitive to microscopic fluctuations and isolated particules. To robustly capture the macroscopic structures we will use the **Cubical Complex** method.

Unlike the Vietoris-Rips this method has to be done for a density on a grid. To track the topological features of the density, we use a superlevel set filtration.

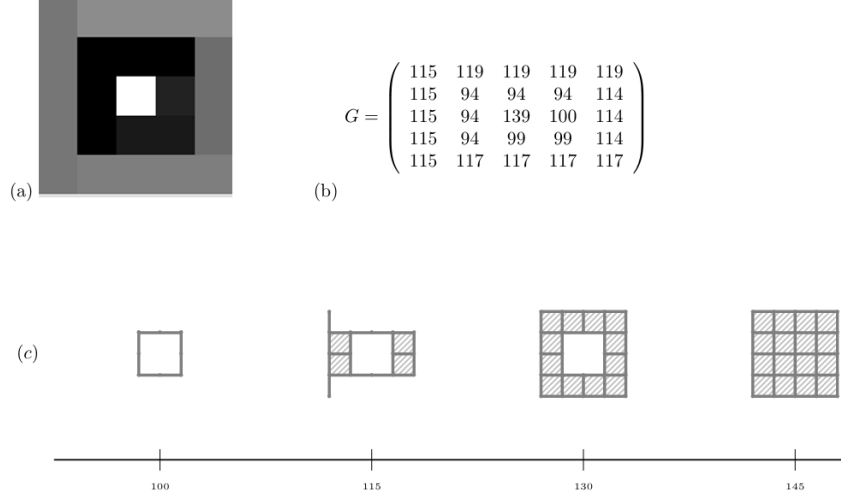


Figure 9: barcode and landscapes

Let $\rho(x, y)$ be the local discrete density of active agents. For a given density threshold ϵ , the superlevel set is the region of the domain where the density is greater than or equal to the threshold:

$$\mathcal{X}_\epsilon = \{(x, y) \in \text{Grid} \mid \rho(x, y) \geq \epsilon\} \quad (5)$$

This filtration is detailed in [1] in subsection 2.2. Instead of growing balls around points, the filtration parameter ϵ starts at the global maximum density and progressively decreases to zero.

To each superlevel set we count the number of connected components as explain in the figure which come from [2].

Computationally we will take $-\rho$ and apply functions from gudhi library where everything is implemented.

Vectorization via Betti Curves Instead of Persistence Landscapes, we vectorize the barcodes using Betti Curves. The Betti curve $\beta_0(\epsilon)$ is a step function defined simply as the rank (or Betti number) of the homology group at a given threshold ϵ so :

$$\beta_0(\epsilon) = \text{rank}(H_0(X_\epsilon)) \quad (6)$$

This curve directly counts the number of independent clusters existing at the density threshold ϵ . The following is the same as before.

3.2 comparison on test

3.2.1 typical patterns

First, I generate 6 typical patterns to compare how those 3 distances are differentiating them. I made an heatmap which is on the github testing all the pairs and writing the distance associated. We have uniform state, large bandes, thin bandes, trails, big clusters, little clusters.

We are making a little analysis of the result for each distance.

- **TDA** : Lower distance is between uniform state and large bandes, second is between little clusters and large clusters which is positive because we want to have them in the same type of pattern. However it is making a significant distance between large bandes and thin bandes.
- **Fourier** : Two smallest distance are between large bandes and uniform state then between small clusters and large clusters. It seems to work the same as TDA except that it captures less the difference between large bandes and trails but this is not an issue because cells orientation is an easy way to differentiate those two.
- **Wasserstein** : For wasserstein distance, in average the ratio between the smallest distance and the higher distance is two times more than the two other measures. Wasserstein quite not see the difference between trails and thin bandes and between uniform states and large bandes. It is penalising more than the others the difference between small clusters and large clusters.

Conclusion : Finally, all distance are little between large bandes and uniform state which is not an issue because we can see on the image that the bands clearly exists but are not very distinct. Each one have it's own specificity, for the next test, huge distance between small clusters and huge clusters will be an issue but as showed the heatmap on other patterns wasserstein could be more efficient.

3.2.2 Emergence of difference between adhesion and repulsion

We try to find a parameter region where at the beginning patterns are the same and when change a set of parameter patterns are changing. To compare patterns difference we will use the three distances explained before.

After unsuccessfully changing only the stiffness or trying to vary the parameter epsilon over the scaling assumption of macro-regime, a test that worked pretty well was to keep to ratio constant :

- $\kappa\mu$: This ratio called b_p in the article played the key role of pattern emergence. We had to keep it less than 1 to see pattern emergence.
- $\frac{\mu}{\zeta}$: This ratio is the term in front of agent-agent force, we can think that keeping it constant could help keeping the same patterns for different parameters but there is no rigorous proof...

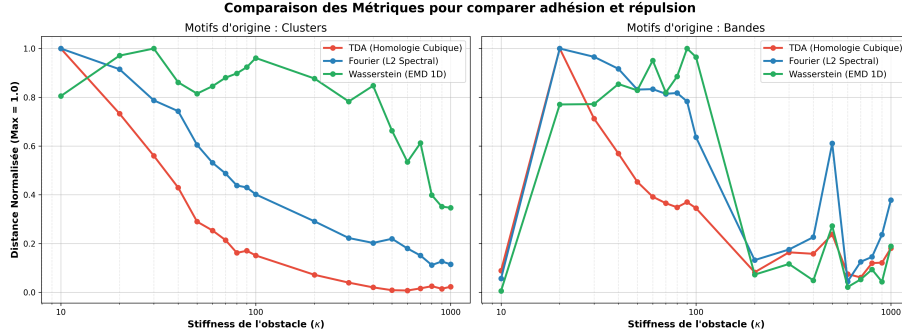


Figure 10: barcode and landscapes

We start at $\kappa = 1000$ on a regime where agents are making bandes for one and clusters for another, then we are reducing the $\kappa = 10$. Images of the end are in the file "test" then "keeping ratio". To describe quickly the results :

- for bandes for long time we have stable bandes for both for long time, then for $\kappa \leq 100$ we can see instabilities emerging creating clusters than captures agents and obstacles around them so the final result is extremely different between adhesive and repulsive and this what we expect to see for the 3 distances (low distance for $200 \leq \kappa \leq 1000$ and huge for $\kappa \leq 100$).
- for clusters at the beggining it is very similar and then for repulsive case it is slightly becoming trails (pinned clusters then moving clusters larger and larger and then connected trails) so we want to observe a progressive augmentation for κ from 1000 to 10.

We plot a comparison between the three distances here

An analysis is that for $\kappa = 100$ clusters are very small in adhesive case and huge in repulsive case (but still clusters on both sides), and wasserstein is penalising it more than comparing trails and large clusters. for the others distances results are encouraging.

For bandes, $\kappa = 10$ is uniform state for both so the three distances are describing it well. The distance deacreasing for TDA does not correspond at what a human eye will see.

Distances must be very high for κ between 10 and 100 and very low between 200 and 1000, Wasserstein is the best distance to show this.

4 Focus on fourier spectrum for pattern recognition

4.1 Get the good informations from the spectrum

The strategy which was decided was to extract the informations in two steps, first keep only the relevant pixels with a well choosen cut-off to avoid the noise. Then to make an elliptic mask of the data we kept and to get relevant quantifier from the ellips.

4.1.1 Relevant cut-off

We are thinking of a rationnal way to make a cut-off of the pixels in the spectrum to avoid noise whithout loosing informations. For now, I generated 200 random configurations, took the fourier spectrum and defined the cut-off equal to the higher maximum value over all the configurations. It seemed to work well to start having good results but needs to be justified more. I finally took $1e6$ as the cut-off.

4.1.2 Ellips mask

We are then calculating the semi-major axis, semi-minor axis and the angle of semi-major axis of an elliptic approximation of the datas. To do it so we calculate order 1 moments :

$$\bar{x} = \frac{M_{10}}{M_{00}} = \frac{\sum x \cdot I(x, y)}{\sum I(x, y)} \quad ; \quad \bar{y} = \frac{M_{01}}{M_{00}} = \frac{\sum y \cdot I(x, y)}{\sum I(x, y)}$$

Then we use the central moments define as :

$$\mu_{pq} = \sum_x \sum_y (x - \bar{x})^p (y - \bar{y})^q I(x, y)$$

To calculate the covariance matrice :

$$C = \frac{1}{M_{00}} \begin{pmatrix} \mu_{20} & \mu_{11} \\ \mu_{11} & \mu_{02} \end{pmatrix}$$

Calling the eigenvalues λ_{max} and λ_{min} we get the semi-major axis $a = k \cdot \sqrt{\lambda_{max}}$ and the semi-minor axis $b = k \cdot \sqrt{\lambda_{min}}$ where k is number of std, we take 3 here to have quite all the pixels in the ellips.

To have the angle we get easily the eigenvectors of C which give the orientatio of both axis then get the angle by taking the arctan.

Finally the excentricity is defined as : $e = \sqrt{1 - \frac{\lambda_{min}}{\lambda_{max}}}$.

4.2 Application on relevant datas

Quick correction on section 3.2.2, after launching simulations for a longer time for bandes, instabilities appears and we can observe a mix of clusters and bandes

at the end of the simulation (the state is not stable), but it is still interesting to see the result of the fourier method described before on such datas.

we can see that it is still working very efficiently on true simulated datas, same

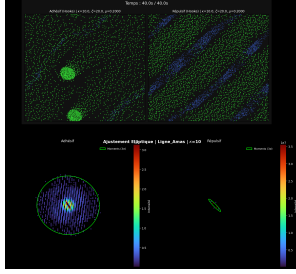


Figure 11: Amas ($\kappa=10$)

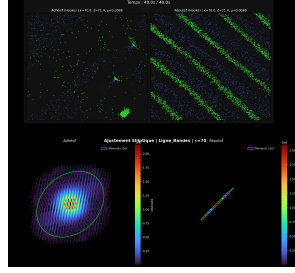


Figure 12: Bandes ($\kappa=70$)

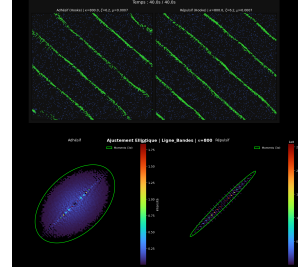


Figure 13: Bandes ($\kappa=800$)

as for synthetical datas and an interesting point is that for mixing patterns we can see the pixels for bandes and the shape of the cluster however we loose this information with the ellips.

References

- [1] Paweł Dłotko and Thomas Wanner. Rigorous cubical approximation and persistent homology of continuous functions. *arXiv preprint arXiv:1610.03833*, 2016.
- [2] Nina Otter, Mason A. Porter, Ulrike Tillmann, Peter Grindrod, and Heather A. Harrington. A roadmap for the computation of persistent homology. *EPJ Data Science*, 2017.
- [3] Thevasha Sathiyakumar et al. Detecting transitions between collective motion regimes using functional hypothesis test of the time-varying persistence homology. *arXiv preprint arXiv:2403.03428*, 2024.